COMPUTER NETWORKS

Week 6

The Transport Layer: TCP (Transmission Control Protocol), UDP (User Datagram Protocol), Ports, Reliable Transfer and Congestion

Lecture • Seminar • Laboratory (Windows and Linux tracks)

Contents

Learning objectives and position in the course

Part I — Lecture

1. Role and services of the transport layer
2. Transport protocols
3. Port numbers, sockets and multiplexing
4. UDP — User Datagram Protocol
5. TCP — Transmission Control Protocol
6. TCP (Transmission Control Protocol) mechanisms: three-way handshake, stop-and-wait, sliding window
7. Traffic congestion and TCP (Transmission Control Protocol) congestion control
8. Summary and key terms

Part II — Seminar: solved problems and exercises

Part III — Laboratory

Lab 6-W: Transport layer on Windows

Lab 6-L: Transport layer on Linux

Lab report template and references

Learning objectives and position in the course

Weeks 6 and 7 together cover the transport layer and its programming interface. **This week (Week 6)** concentrates on the transport layer itself: what services it offers, how UDP (User Datagram Protocol) and TCP (Transmission Control Protocol) differ, how port numbers identify processes, how TCP (Transmission Control Protocol) opens and closes connections, how it delivers data reliably (stop-and-wait and sliding-window transmission) and how it reacts to traffic congestion. **Next week (Week 7)** builds on this foundation with sockets and Java client-server programming over TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

Table 1. *How the transport-layer chapter is split across Weeks 6 and 7*

Week	Lecture topics	Seminar	Laboratory
6 (this week)	Transport protocols; UDP (User Datagram Protocol); TCP (Transmission Control Protocol); port numbers; three-way handshake; stop-and-wait; sliding window; congestion	Port/socket identification, efficiency of stop-and-wait vs sliding window, sequence/ACK (Acknowledgment) tracing, congestion window traces, header decoding	Inspecting ports and connections, capturing the handshake, UDP (User Datagram Protocol) vs TCP (Transmission Control Protocol) traffic, TCP (Transmission Control Protocol) settings and congestion control (Windows and Linux)
7 (next week)	Sockets: definitions, classification, socket API (Application Programming Interface); Java TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) client-server programming	Socket call sequences, framing, debugging socket code, protocol design	Building and testing Java TCP/UDP (Transmission Control Protocol/User Datagram Protocol) clients and servers (Windows and Linux)

By the end of this week, students should be able to:

1. Explain the role of the transport layer and the difference between process-to-process and host-to-host delivery.
2. Compare TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) and choose the appropriate protocol for a given application.
3. Use port numbers and the 5-tuple to explain multiplexing and demultiplexing; classify ports into well-known, registered and dynamic ranges.
4. Describe the UDP (User Datagram Protocol) and TCP (Transmission Control Protocol) headers and interpret their fields in a packet capture.
5. Trace a TCP (Transmission Control Protocol) connection establishment (three-way handshake) and termination, including sequence and acknowledgment numbers.
6. Compute the efficiency of stop-and-wait and sliding-window transmission and size a window from the bandwidth-delay product.
7. Explain the causes of congestion and the TCP (Transmission Control Protocol) congestion-control algorithms (slow start, congestion avoidance, fast retransmit/fast recovery).
8. Use operating-system tools (netstat/ss, PowerShell, tcpdump, Wireshark) to observe transport-layer behaviour on Windows and Linux.

Part I — Lecture

1. Role and services of the transport layer

The network layer (IP (Internet Protocol)) delivers packets **from host to host**, on a best-effort basis: packets may be lost, duplicated, delayed or delivered out of order. Applications, however, run as **processes**, and a host typically runs many communicating processes at once (a browser, a mail client, a DNS (Domain Name System) resolver, an SSH (Secure Shell) session). The transport layer closes this gap: it provides **logical, end-to-end communication between processes** running on different hosts.

The transport layer is implemented only in the **end systems** (hosts), usually inside the operating-system kernel. Routers in the middle of the network process packets only up to the network layer and do not look at transport-layer state. The transport-layer data unit is called a **segment** in TCP (Transmission Control Protocol) and a **datagram** in UDP (User Datagram Protocol); it is carried as the payload of an IP (Internet Protocol) packet.

Table 2. *Services that a transport protocol may offer*

Service	What it means	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Multiplexing / demultiplexing	Many processes share one IP (Internet Protocol) address; ports deliver data to the right process	Yes	Yes
Error detection	Checksum over header + data (+ IP (Internet Protocol) pseudo-header)	Yes (optional in IPv4 (Internet Protocol version 4), mandatory in IPv6 (Internet Protocol version 6))	Yes (mandatory)
Reliable delivery	No loss, no duplicates, in-order delivery	No	Yes (ACKs (Acknowledgments), retransmissions, sequence numbers)
Connection management	Explicit set-up and tear-down of a session	No (connectionless)	Yes (three-way handshake, FIN (Finish) exchange)
Flow control	Sender does not overrun the receiver's buffer	No	Yes (receiver window, rwnd)
Congestion control	Sender does not overload the network	No	Yes (congestion window, cwnd)
Timing / bandwidth guarantees	Bounded delay or guaranteed throughput	No	No

> **Key idea**
> The transport layer turns IP (Internet Protocol)'s unreliable, host-to-host packet delivery into a service that applications can use directly. UDP (User Datagram Protocol) adds only the minimum (ports and a checksum); TCP (Transmission Control Protocol) adds a full reliable byte-stream service on top of an unreliable network.

2. Transport protocols

The TCP/IP (Transmission Control Protocol/Internet Protocol) suite defines several transport protocols. Two of them carry almost all Internet traffic: **UDP** (User Datagram Protocol; RFC (Request for Comments) 768,

1980) and **TCP** (Transmission Control Protocol; originally RFC (Request for Comments) 793, 1981; consolidated in RFC (Request for Comments) 9293, 2022). Others exist for specialised needs.

Table 3. *Overview of transport protocols*

Protocol	Type	Main features	Typical use
UDP (User Datagram Protocol)	Connectionless, message-oriented	8-byte header, no reliability, no ordering, preserves message boundaries, supports broadcast/multicast	DNS (Domain Name System), DHCP (Dynamic Host Configuration Protocol), SNMP (Simple Network Management Protocol), TFTP (Trivial File Transfer Protocol), NTP (Network Time Protocol), VoIP (Voice over Internet Protocol)/video (RTP (Real-time Transport Protocol)), online games, QUIC (Quick UDP Internet Connections)
TCP (Transmission Control Protocol)	Connection-oriented, byte-stream	Reliable, ordered, full-duplex, flow control, congestion control	HTTP/1.1 (Hypertext Transfer Protocol) and HTTP/2 (Hypertext Transfer Protocol), HTTPS (Hypertext Transfer Protocol Secure; TLS (Transport Layer Security)), SMTP (Simple Mail Transfer Protocol), IMAP (Internet Message Access Protocol), SSH (Secure Shell), FTP (File Transfer Protocol), databases
SCTP (Stream Control Transmission Protocol)	Connection-oriented, message-oriented	Multi-streaming, multi-homing, reliable messages	Telecom signalling (SIGTRAN (Signalling Transport)), WebRTC (Web Real-Time Communication) data channels
DCCP (Datagram Congestion Control Protocol)	Connection-oriented, unreliable	Congestion control without reliability	Experimental streaming media
QUIC (Quick UDP Internet Connections)	Runs over UDP (User Datagram Protocol), implemented in user space	Encrypted (TLS (Transport Layer Security) 1.3), streams without head-of-line blocking, 0-RTT/1-RTT (0 Round-Trip Time/1 Round-Trip Time) setup, connection migration	HTTP/3 (Hypertext Transfer Protocol), modern browsers and CDNs (Content Delivery Networks)

When to choose UDP (User Datagram Protocol)? When the application prefers timeliness over reliability (real-time voice and video), when exchanges are a single short request and reply (DNS (Domain Name System)), when broadcast or multicast is needed, or when the application wants to implement its own reliability and congestion control (QUIC (Quick UDP Internet Connections)). **When to choose TCP (Transmission Control Protocol)?** When every byte must arrive exactly once and in order (web pages, files, e-mail, remote shells) and the application prefers the operating system to handle reliability.

3. Port numbers, sockets and multiplexing

A **port number** is a 16-bit unsigned integer (0–65535) that identifies a communication endpoint inside a host. Combined with an IP (Internet Protocol) address and a transport protocol it identifies a specific process (more precisely, a specific socket opened by a process).

Table 4. *Port number ranges (IANA (Internet Assigned Numbers Authority), RFC (Request for Comments) 6335)*

Range	Name	Assignment	Examples
0 – 1023	Well-known (system) ports	Assigned by IANA (Internet Assigned Numbers Authority) to standard services; on Linux/Unix binding to them requires administrative privileges	20/21 FTP (File Transfer Protocol), 22 SSH (Secure Shell), 23 Telnet, 25 SMTP (Simple Mail Transfer Protocol), 53 DNS (Domain Name System), 67/68 DHCP (Dynamic Host Configuration Protocol), 80 HTTP (Hypertext Transfer Protocol), 110 POP3 (Post Office Protocol version 3), 123 NTP (Network Time Protocol), 143 IMAP (Internet Message Access Protocol), 161 SNMP (Simple Network Management Protocol), 443 HTTPS (Hypertext Transfer Protocol Secure)
1024 – 49151	Registered (user) ports	Registered with IANA (Internet Assigned Numbers Authority) for specific applications	1433 MS (Microsoft) SQL (Structured Query Language), 1883 MQTT (Message Queuing Telemetry Transport), 3306 MySQL, 3389 RDP (Remote Desktop Protocol), 5432 PostgreSQL, 8080 HTTP (Hypertext Transfer Protocol)-alt
49152 – 65535	Dynamic / private / ephemeral ports	Never assigned; used by the OS (Operating System) for the client side of connections	Windows default ephemeral range 49152 –65535; Linux default 32768–60999

3.1 Socket address and the 5-tuple

A **socket address** is the pair (IP (Internet Protocol) address, port), for example `192.168.1.10:51544`. A **TCP (Transmission Control Protocol) connection** is uniquely identified by the **5-tuple** (protocol, source IP (Internet Protocol), source port, destination IP (Internet Protocol), destination port). Because the tuple includes the client side, a web server can handle thousands of connections on the same local port 443 at the same time: each connection differs at least in the client address or client port.

3.2 Demultiplexing

- **UDP (User Datagram Protocol) demultiplexing** uses only the destination (IP (Internet Protocol), port). All datagrams sent to `10.0.0.5:53` go to the same socket, regardless of the sender; the application learns the sender from the received datagram.
- **TCP (Transmission Control Protocol) demultiplexing** uses the full 5-tuple. A server has one *listening socket* (e.g. `*:443`) plus one *connected socket* per client; incoming segments are delivered to the connected socket whose 5-tuple matches.
- **Multiplexing** is the reverse operation at the sender: data from many sockets is gathered, given transport headers with the correct ports, and passed down to IP (Internet Protocol).

Example

A laptop `192.168.1.10` opens two browser tabs to the same server `93.184.215.14:443`. The OS (Operating System) assigns two ephemeral ports, e.g. 51544 and 51545. The two connections are

(TCP (Transmission Control Protocol), 192.168.1.10, 51544, 93.184.215.14, 443) and (TCP (Transmission Control Protocol), 192.168.1.10, 51545, 93.184.215.14, 443).

They share the destination socket address but are different connections, so the server's TCP (Transmission Control Protocol) keeps them apart.

4. UDP — User Datagram Protocol

UDP (User Datagram Protocol) is the simplest possible transport protocol: it adds to IP (Internet Protocol) only port-based multiplexing and an optional error check. Each `send` operation produces exactly one UDP (User Datagram Protocol) datagram, and each `receive` returns exactly one datagram: **message boundaries are preserved**. There is no connection set-up, no acknowledgment, no retransmission, no ordering and no congestion control.

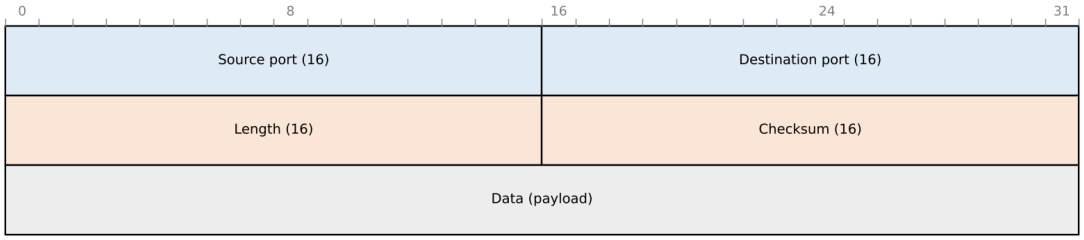


Figure 1. *UDP (User Datagram Protocol) header format (8 bytes)*

Table 5. *UDP (User Datagram Protocol) header fields*

Field	Size	Meaning
Source port	16 bits	Port of the sending process (may be 0 if no reply is expected)
Destination port	16 bits	Port of the receiving process
Length	16 bits	Length of header + data in bytes (minimum 8); payload max 65,507 bytes over IPv4 (Internet Protocol version 4)
Checksum	16 bits	One's-complement sum over a pseudo-header (source/destination IP (Internet Protocol), protocol 17, UDP (User Datagram Protocol) length), the UDP (User Datagram Protocol) header and the data. 0 = not computed (IPv4 (Internet Protocol version 4) only)

Advantages: no connection delay, small header (8 bytes vs at least 20 for TCP (Transmission Control Protocol)), no connection state on the server (scales to many clients), the application controls exactly what is sent and when. **Disadvantages:** losses, duplicates and reordering must be handled by the application if they matter; UDP (User Datagram Protocol) senders that do not control their rate can harm other traffic.

5. TCP — Transmission Control Protocol

TCP (Transmission Control Protocol) provides a **reliable, ordered, full-duplex byte stream** between two processes. The application writes a stream of bytes; TCP (Transmission Control Protocol) splits it into **segments**, numbers every byte, retransmits what is lost and delivers the bytes in order at the other end. TCP (Transmission Control Protocol) does **not** preserve message boundaries: two `write` calls of 5 bytes may arrive as one `read` of 10 bytes, or as reads of 3 and 7 bytes.

- **Connection-oriented:** an explicit handshake creates state at both ends before data flows.
- **Point-to-point and full-duplex:** exactly two endpoints; data flows in both directions simultaneously.
- **Reliable:** sequence numbers, cumulative acknowledgments, checksums, retransmission timers.
- **Flow control:** the receiver advertises the free space in its buffer (the *receive window*).
- **Congestion control:** the sender limits its rate according to the estimated network load (the *congestion window*).

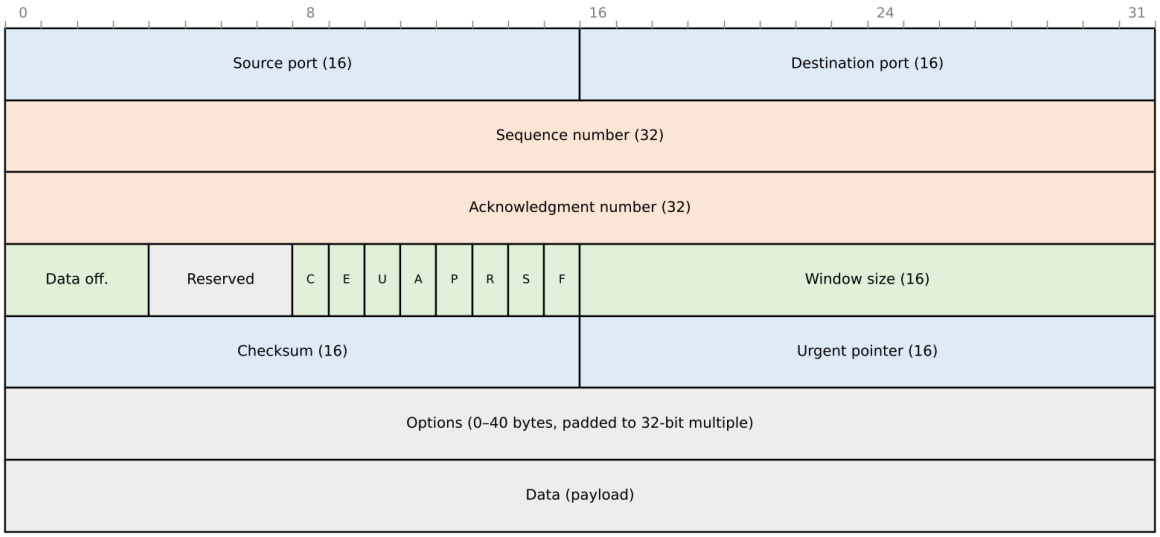


Figure 2. *TCP (Transmission Control Protocol) header format (20 bytes without options, maximum 60 bytes)*

Table 6. *TCP (Transmission Control Protocol) header fields*

Field	Size	Meaning
Source / destination port	16 + 16 bits	Identify the sending and receiving sockets
Sequence number	32 bits	Number of the first data byte in this segment. In a SYN (Synchronize) it carries the Initial Sequence Number (ISN)
Acknowledgment number	32 bits	Next byte the sender of this segment expects to receive (cumulative ACK (Acknowledgment)); valid when ACK (Acknowledgment) flag = 1
Data offset	4 bits	Header length in 32-bit words (5 to 15, i.e. 20 to 60 bytes)
Flags	8 bits	CWR (Congestion Window Reduced), ECE (ECN-Echo; explicit congestion notification), URG (Urgent), ACK (Acknowledgment), PSH (Push), RST (reset/abort), SYN (Synchronize; synchronise / open), FIN (finish / close)
Window size	16 bits	Receive window: how many bytes, beyond the acknowledged one, the receiver can accept (scaled by the Window Scale option)
Checksum	16 bits	Covers pseudo-header, TCP (Transmission Control Protocol) header and data (mandatory)
Urgent pointer	16 bits	Offset of urgent data when URG (Urgent) = 1 (rarely used)
Options	0–40 bytes	MSS (Maximum Segment Size), Window Scale, SACK (Selective Acknowledgment)-permitted / SACK (Selective Acknowledgment) blocks, Timestamps

The **Maximum Segment Size (MSS)** is the largest amount of *data* in one segment. On Ethernet (MTU (Maximum Transmission Unit) 1500 bytes) with IPv4 (Internet Protocol version 4) and no options, MSS (Maximum Segment Size) = 1500 − 20 (IP (Internet Protocol)) − 20 (TCP (Transmission Control Protocol)) = **1460 bytes**. Each side announces its MSS (Maximum Segment Size) in the SYN (Synchronize) segment.

6. TCP (Transmission Control Protocol) mechanisms

6.1 Connection establishment: the three-way handshake

Before data is exchanged, both ends must agree that a connection exists and must learn each other's **Initial Sequence Number (ISN)**, window size and options. TCP (Transmission Control Protocol) does this with three segments:

1. **SYN (Synchronize)** — the client (active opener) sends a segment with SYN (Synchronize) = 1 and `seq = x` (its ISN (Initial Sequence Number)). The client moves to state SYN_SENT.
2. **SYN (Synchronize) + ACK (Acknowledgment)** — the server (passive opener, in state LISTEN) replies with SYN (Synchronize) = 1, ACK (Acknowledgment) = 1, `seq = y` (its own ISN (Initial Sequence Number)) and `ack = x + 1`. The server moves to SYN_RCVD.
3. **ACK (Acknowledgment)** — the client acknowledges with ACK (Acknowledgment) = 1, `seq = x + 1`, `ack = y + 1`. Both ends are now ESTABLISHED and data may flow.

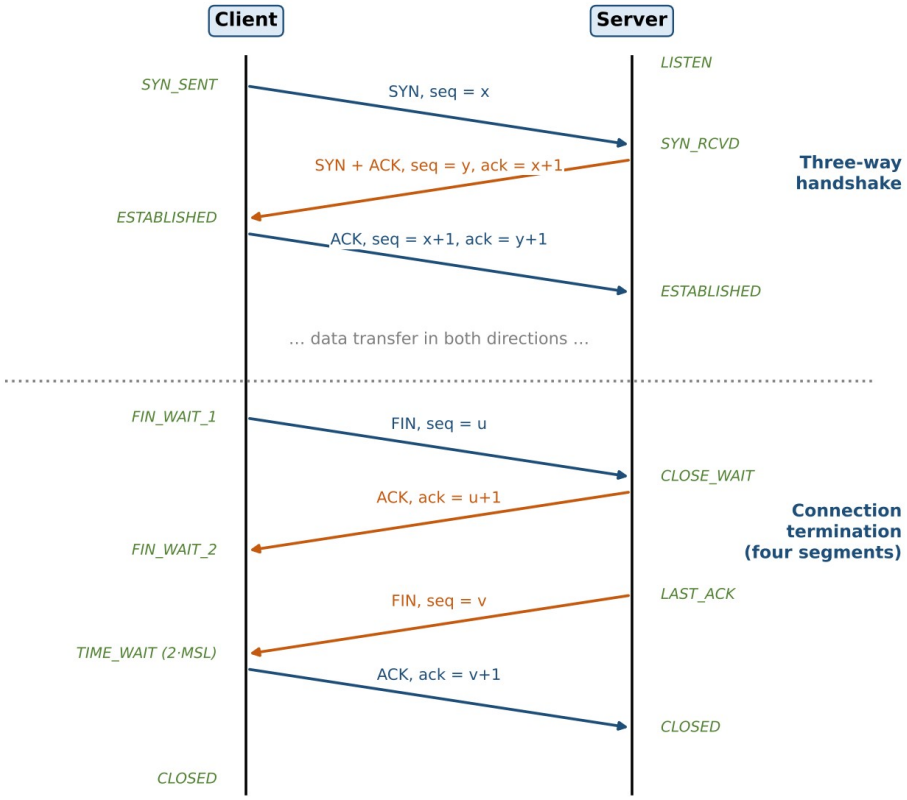


Figure 3. *TCP (Transmission Control Protocol) three-way handshake and four-segment connection termination, with the TCP (Transmission Control Protocol) states of each end*

Why three segments and not two? Each side must have its own ISN (Initial Sequence Number) acknowledged. Two segments would leave the server unsure whether the client received its SYN (Synchronize), and a delayed duplicate SYN (Synchronize) from an old connection could open a bogus connection. The SYN (Synchronize) and FIN (Finish) flags each consume **one sequence number**, which is why the acknowledgments carry x + 1 and y + 1.

Why random ISNs (Initial Sequence Numbers)? ISNs (Initial Sequence Numbers) are chosen pseudo-randomly (RFC (Request for Comments) 6528) so that segments from an old connection with the same 5-tuple are unlikely to fall inside the new connection's sequence space, and so that attackers cannot easily guess sequence numbers to inject data.

6.2 Connection termination and TCP (Transmission Control Protocol) states

Since TCP (Transmission Control Protocol) is full-duplex, each direction is closed independently. The side that closes first sends **FIN (Finish)**; the other side acknowledges it, may still send data (half-closed connection), and later sends its own FIN (Finish), which is acknowledged. The side that sent the first FIN (Finish) waits in **TIME_WAIT** for 2·MSL (Maximum Segment Lifetime; in practice 30 s to 4 min, 60 s on Linux) so that a lost final ACK (Acknowledgment) can be retransmitted and old duplicates expire. A connection can also be aborted abruptly with **RST (Reset)** — for example when a SYN (Synchronize) arrives on a port where no process listens.

Table 7. *Main TCP (Transmission Control Protocol) connection states (visible with netstat / ss)*

State	Meaning
LISTEN	Server waits for incoming connection requests
SYN_SENT / SYN_RCVD	Handshake in progress (client / server side)
ESTABLISHED	Connection open; data transfer
FIN_WAIT_1 / FIN_WAIT_2	Active closer has sent FIN (Finish); waiting for ACK (Acknowledgment) and then for the peer's FIN (Finish)
CLOSE_WAIT	Passive closer received FIN (Finish); waits for the local application to close
LAST_ACK	Passive closer has sent its FIN (Finish); waits for the final ACK (Acknowledgment)
TIME_WAIT	Active closer waits 2·MSL (Maximum Segment Lifetime) before releasing the 5-tuple
CLOSED	No connection

Common observation in the lab

Many connections in **CLOSE_WAIT** on a server usually indicate an application bug: the peer has closed, but the program never calls `close()` on its socket. Many connections in **TIME_WAIT** are normal on a busy client or on the side that closes first.

6.3 Reliable transfer: stop-and-wait transmission

The simplest reliable protocol over an unreliable channel is **stop-and-wait ARQ** (Automatic Repeat reQuest). The sender transmits one frame (segment), starts a timer and waits. The receiver checks the frame and returns an acknowledgment. If the ACK (Acknowledgment) arrives before the timer expires, the sender transmits the next frame; otherwise it retransmits the same frame. A **1-bit sequence number** (0, 1, 0, 1, …) is enough to let the receiver recognise duplicates caused by lost ACKs (Acknowledgments).

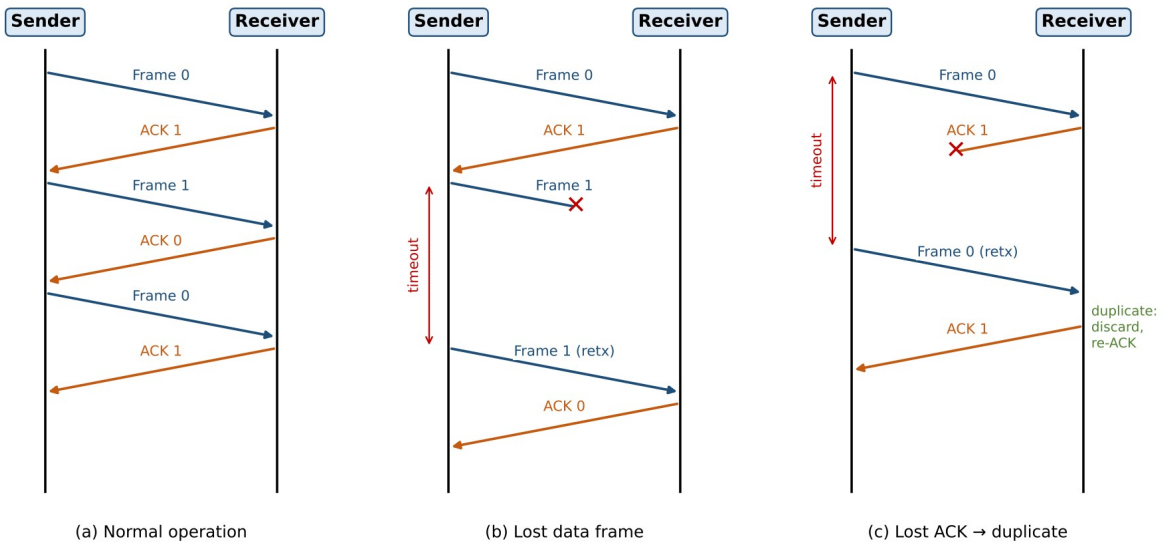


Figure 4. *Stop-and-wait ARQ (Automatic Repeat reQuest): (a) normal operation, (b) lost frame recovered by timeout, (c) lost ACK (Acknowledgment) causing a duplicate that the receiver discards*

Stop-and-wait is correct but inefficient on links with a long delay or a high bitrate, because the sender is idle while waiting for each ACK (Acknowledgment). Let **T_t = L / R** be the transmission time of a frame (L bits on a link of R bit/s) and **RTT (Round-Trip Time)** the round-trip propagation time. If the ACK (Acknowledgment) transmission time is negligible, the **utilisation** of the sender is:

Stop-and-wait utilisation

U = T_t / (T_t + RTT (Round-Trip Time)) = 1 / (1 + 2a), where a = T_prop / T_t

Throughput = U · R

Example: R = 1 Gbit/s, L = 1500 bytes = 12,000 bits, RTT (Round-Trip Time) = 30 ms. Then T_t = 12 µs and U = 0.012 / 30.012 ≈ **0.04 %** — a 1 Gbit/s link carries only about 400 kbit/s. This motivates **pipelining**.

6.4 Sliding-window transmission

With **sliding-window** (pipelined) transmission, the sender may have up to **W** frames sent but not yet acknowledged. The window covers a range of sequence numbers; when the oldest outstanding frame is acknowledged, the window **slides** forward and new frames may be sent. Both sender and receiver keep windows: the sender's window says what may be sent, the receiver's window says what may be accepted.

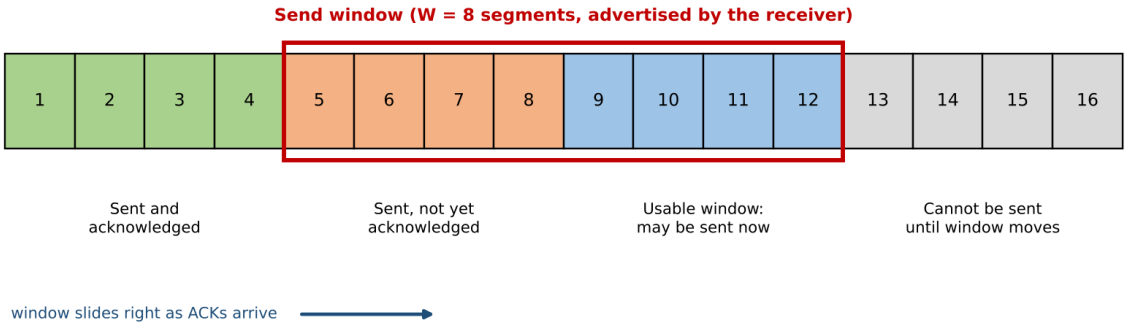


Figure 5. *Sender's view of the sequence space with a sliding window of W = 8 segments*

Sliding-window utilisation

U = min(1, W · T_t / (T_t + RTT (Round-Trip Time)))

To keep the link fully busy: W ≥ (T_t + RTT (Round-Trip Time)) / T_t, i.e. the window must cover the **bandwidth-delay product** R · RTT (Round-Trip Time).

Two classic error-recovery strategies are used with sliding windows:

Table 8. *Go-Back-N vs Selective Repeat*

	Go-Back-N (GBN)	Selective Repeat (SR)
Receiver window	1 — out-of-order frames are discarded	W — out-of-order frames are buffered
Acknowledgments	Cumulative (ACK (Acknowledgment) n = all frames before n received)	Individual, per frame
On loss / timeout	Retransmit the lost frame and all frames sent after it	Retransmit only the lost frame
Sequence numbers (k bits)	W ≤ 2^k − 1	W ≤ 2^(k−1)
Complexity	Simple receiver, wasteful on lossy links	More buffering and logic, efficient

6.5 How TCP (Transmission Control Protocol) uses the sliding window

- TCP (Transmission Control Protocol) numbers **bytes**, not segments; the window is expressed in bytes.

- Acknowledgments are **cumulative** (like GBN (Go-Back-N)), but receivers buffer out-of-order data and the **SACK (Selective Acknowledgment)** option (RFC (Request for Comments) 2018) lets them report exactly which blocks arrived, so the sender retransmits only what is missing (like SR (Selective Repeat)).
- The receiver advertises its free buffer space in the **Window** field (**rwnd**) — this is **flow control**. A receiver with a full buffer advertises a **zero window**; the sender then sends small *window probes* until space is available.
- Because the 16-bit field limits rwnd to 65,535 bytes, the **Window Scale** option (RFC (Request for Comments) 7323) multiplies it by 2^S (S ≤ 14), allowing windows up to about 1 GiB on high bandwidth-delay paths.
- The amount of unacknowledged data the sender may have in flight is **min(rwnd, cwnd)**, where cwnd is the congestion window (Section 7).

6.6 Retransmission timeout and fast retransmit

TCP (Transmission Control Protocol) measures the round-trip time and adapts its **Retransmission Timeout (RTO)** using the algorithm of RFC (Request for Comments) 6298 (α = 1/8, β = 1/4):

RTO (Retransmission Timeout) estimation (RFC (Request for Comments) 6298)

```
first sample R:   SRTT = R;   RTTVAR = R / 2
later samples R:  RTTVAR = (1 - β)·RTTVAR + β·|SRTT - R|
                  SRTT   = (1 - α)·SRTT   + α·R
                  RTO    = SRTT + max(G, 4·RTTVAR)     (G = clock granularity)
on timeout:       RTO = 2·RTO   (exponential back-off)
```

Waiting for a timeout is slow. If the sender receives **three duplicate ACKs (Acknowledgments)** for the same byte, it concludes that the following segment was lost (later segments are arriving) and retransmits it immediately — **fast retransmit**.

7. Traffic congestion and TCP (Transmission Control Protocol) congestion control

7.1 What is congestion?

Congestion occurs when the total rate at which sources send into some part of the network exceeds the capacity of a link or router. Router queues fill up, packets wait longer (increasing delay), and when buffers overflow packets are **dropped**. Senders then retransmit, adding even more load. In the extreme, the network carries mostly retransmissions and useful throughput collapses — the **congestion collapse** observed on the early Internet in 1986, which led Van Jacobson to design TCP (Transmission Control Protocol) congestion control (1988).

Table 9. *Costs of congestion*

Symptom	Cause	Consequence
Growing queuing delay	Arrival rate close to link capacity	Higher RTT (Round-Trip Time), poor interactivity, possible premature timeouts
Packet loss	Buffer overflow at a router	Retransmissions; wasted capacity on upstream links
Unnecessary retransmissions	Delayed (not lost) packets time out	Duplicate copies use bandwidth
Throughput collapse	Most capacity used by packets dropped later	Network delivers little useful data

Flow control vs congestion control: flow control protects the **receiver** (rwnd, advertised by the receiver); congestion control protects the **network** (cwnd, estimated by the sender). TCP (Transmission Control Protocol) sends at most min(rwnd, cwnd) unacknowledged bytes.

7.2 Approaches

- **End-to-end congestion control** — the network gives no explicit feedback; end systems infer congestion from loss and delay. This is classic TCP (Transmission Control Protocol).
- **Network-assisted congestion control** — routers signal congestion, e.g. **ECN** (Explicit Congestion Notification, RFC (Request for Comments) 3168): a router marks the IP (Internet Protocol) header instead of dropping; the receiver echoes the mark with the TCP (Transmission Control Protocol) ECE (ECN-Echo) flag; the sender reduces cwnd and confirms with CWR (Congestion Window Reduced).
- **Queue management in routers** — Active Queue Management (RED (Random Early Detection), CoDel (Controlled Delay), fq_codel) drops or marks early to keep queues short.

7.3 TCP (Transmission Control Protocol) congestion-control algorithms (Tahoe / Reno, RFC (Request for Comments) 5681)

The sender keeps two variables: the **congestion window cwnd** (in bytes or MSS (Maximum Segment Size) units) and the **slow-start threshold ssthresh**. The principle is **AIMD — Additive Increase, Multiplicative Decrease**: probe gently for more bandwidth while there is no loss, cut back sharply when loss signals congestion.

Table 10. *TCP (Transmission Control Protocol) Reno congestion-control rules*

Phase / event	Rule	Effect
Slow start (cwnd < ssthresh)	cwnd += 1 MSS (Maximum Segment Size) for every ACK (Acknowledgment) received	cwnd doubles every RTT (Round-Trip Time; exponential growth); initial cwnd is up to 10 MSS (Maximum Segment Size) on modern systems (RFC (Request for Comments) 6928)
Congestion avoidance (cwnd ≥ ssthresh)	cwnd += MSS (Maximum Segment Size)·MSS (Maximum Segment Size)/cwnd per ACK (Acknowledgment)	cwnd grows by ≈ 1 MSS (Maximum Segment Size) per RTT (Round-Trip Time; additive increase)
3 duplicate ACKs (Acknowledgments; Reno)	ssthresh = cwnd/2; fast retransmit; cwnd = ssthresh (+3 MSS (Maximum Segment Size) during fast recovery)	Moderate reaction: the network still delivers packets
Retransmission timeout	ssthresh = cwnd/2; cwnd = 1 MSS (Maximum Segment Size); back to slow start	Strong reaction: severe congestion assumed
TCP (Transmission Control Protocol) Tahoe (older)	Any loss → cwnd = 1 MSS (Maximum Segment Size), slow start	No fast recovery

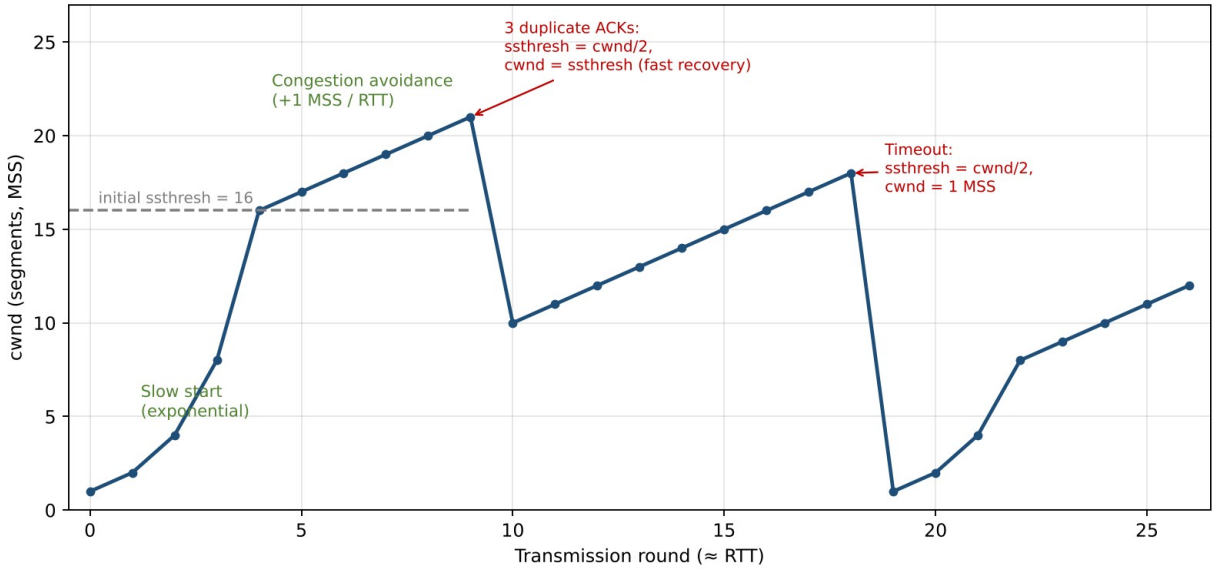


Figure 6. *Evolution of cwnd for TCP (Transmission Control Protocol) Reno: slow start, congestion avoidance, fast recovery after three duplicate ACKs (Acknowledgments; round 9) and a timeout (round 18)*

7.4 Modern algorithms

- **CUBIC** (RFC (Request for Comments) 9438) — cwnd grows as a cubic function of the time since the last loss, which works much better on high bandwidth-delay paths. Default on Linux since kernel 2.6.19 and on Windows since Windows 10 version 1709 / Server 2019.
- **Compound TCP (Transmission Control Protocol)** — older Windows default that combines loss- and delay-based windows.
- **BBR** (Bottleneck Bandwidth and Round-trip propagation time; Google) — model-based: estimates bottleneck bandwidth and minimum RTT (Round-Trip Time) and paces packets accordingly instead of reacting only to loss. Available on Linux (`tcp_bbr` module).
- **QUIC (Quick UDP Internet Connections)** implementations run congestion control in user space (typically CUBIC or BBR (Bottleneck Bandwidth and Round-trip propagation time)).

Fairness: with AIMD (Additive Increase, Multiplicative Decrease), N TCP (Transmission Control Protocol) flows sharing a bottleneck converge towards an equal share R/N each. Applications can bypass fairness by opening many parallel connections, and UDP (User Datagram Protocol) applications without congestion control are not constrained at all — one reason why real-time applications are expected to implement their own rate control.

8. Summary and key terms

- The transport layer provides process-to-process communication; it runs only in end systems.
- Ports (16 bits) plus IP (Internet Protocol) addresses identify sockets; a TCP (Transmission Control Protocol) connection is identified by its 5-tuple.
- UDP (User Datagram Protocol): connectionless, 8-byte header, message-oriented, no reliability — fast and simple.
- TCP (Transmission Control Protocol): connection-oriented reliable byte stream, 20–60-byte header, sequence numbers count bytes.
- Three-way handshake (SYN (Synchronize), SYN-ACK (Synchronize-Acknowledgment), ACK (Acknowledgment)) synchronises ISNs (Initial Sequence Numbers); FIN/ACK (Finish/Acknowledgment) exchange closes each direction; TIME_WAIT protects against old duplicates.
- Stop-and-wait is simple but inefficient (U = T_t/(T_t + RTT (Round-Trip Time))); sliding windows fill the pipe when W ≥ bandwidth-delay product.

- Flow control (rwnd) protects the receiver; congestion control (cwnd: slow start, congestion avoidance, fast retransmit/recovery, AIMD (Additive Increase, Multiplicative Decrease)) protects the network.

Key terms: segment, datagram, port, ephemeral port, socket address, 5-tuple, multiplexing, demultiplexing, ISN (Initial Sequence Number), SYN (Synchronize), ACK (Acknowledgment), FIN (Finish), RST (Reset), MSS (Maximum Segment Size), MTU (Maximum Transmission Unit), RTT (Round-Trip Time), RTO (Retransmission Timeout), ARQ (Automatic Repeat reQuest), Go-Back-N, Selective Repeat, SACK (Selective Acknowledgment), rwnd, cwnd, ssthresh, AIMD (Additive Increase, Multiplicative Decrease), slow start, fast retransmit, fast recovery, ECN (Explicit Congestion Notification), CUBIC, BBR (Bottleneck Bandwidth and Round-trip propagation time).

Part II — Seminar: solved problems and exercises

The seminar applies the lecture concepts to numerical and tracing problems. Work through the solved problems in class; the proposed exercises are for individual work and homework.

Problem 1 — Ports, sockets and demultiplexing

Host A (`10.0.0.5`) runs a web server on TCP (Transmission Control Protocol) port 80 and a DNS (Domain Name System) server on UDP (User Datagram Protocol) port 53. Host B (`10.0.0.20`) opens two TCP (Transmission Control Protocol) connections to the web server from ports 50001 and 50002, and sends a DNS (Domain Name System) query from UDP (User Datagram Protocol) port 50001. Host C (`10.0.0.30`) opens one TCP (Transmission Control Protocol) connection to the web server, also from port 50001.

1. How many sockets does Host A use, and what identifies each one?
2. Can B use port 50001 for TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) at the same time? Can B and C both use client port 50001?
3. Which fields does Host A use to deliver an incoming segment to the right socket?

> **Solution**
>
> 1. Host A uses **5 sockets**: 1 TCP (Transmission Control Protocol) listening socket `*:80`, 3 TCP (Transmission Control Protocol) connected sockets — (10.0.0.5:80, 10.0.0.20:50001), (10.0.0.5:80, 10.0.0.20:50002), (10.0.0.5:80, 10.0.0.30:50001) — and 1 UDP (User Datagram Protocol) socket `*:53`.
>
> 2. Yes: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) have separate port spaces. Yes: the connections from B and C differ in the source IP (Internet Protocol), so their 5-tuples are different.
>
> 3. For TCP (Transmission Control Protocol): the full 5-tuple (protocol, source IP (Internet Protocol), source port, destination IP (Internet Protocol), destination port). For UDP (User Datagram Protocol): only the destination IP (Internet Protocol) and destination port.

Problem 2 — Efficiency of stop-and-wait

A link has a bitrate R = 10 Mbit/s and a one-way propagation delay of 10 ms. Frames carry L = 1250 bytes. ACKs (Acknowledgments) are very short (their transmission time can be neglected). Compute the utilisation and the effective throughput with stop-and-wait.

> **Solution**
>
> T_t = L / R = 10,000 bits / 10^7 bit/s = **1 ms**. RTT (Round-Trip Time) = 2 · 10 ms = **20 ms**.
>
> U = T_t / (T_t + RTT (Round-Trip Time)) = 1 / 21 ≈ **4.76 %**.
>
> Throughput = U · R ≈ 0.0476 · 10 Mbit/s ≈ **476 kbit/s**.

Problem 3 — Sliding window on the same link

Using the link from Problem 2: (a) What window size W (in frames) is needed for 100 % utilisation? (b) What are the utilisation and throughput with W = 8? (c) How many sequence-number bits are needed for the window from (a) with Go-Back-N and with Selective Repeat?

Solution

(a) W ≥ (T_t + RTT (Round-Trip Time)) / T_t = 21 / 1 = **21 frames**.

(b) U = 8 · 1 / 21 ≈ **38.1 %**; throughput ≈ **3.81 Mbit/s**.

(c) Go-Back-N: W ≤ 2^k − 1 → 2^k ≥ 22 → **k = 5 bits**. Selective Repeat: W ≤ 2^(k−1) → 2^(k−1) ≥ 21 → **k = 6 bits**.

Problem 4 — Bandwidth-delay product and the TCP (Transmission Control Protocol) window

A TCP (Transmission Control Protocol) connection runs over a path with R = 100 Mbit/s and RTT (Round-Trip Time) = 80 ms. (a) Compute the bandwidth-delay product. (b) What is the maximum throughput if the receive window is limited to 65,535 bytes (no window scaling)? (c) What Window Scale shift is needed to fill the pipe?

Solution

(a) BDP (Bandwidth-Delay Product) = R · RTT (Round-Trip Time) = 10^8 · 0.08 = 8 · 10^6 bits = **1,000,000 bytes**.

(b) At most one window per RTT (Round-Trip Time): 65,535 · 8 bits / 0.08 s ≈ **6.55 Mbit/s** (only 6.5 % of the link).

(c) 1,000,000 / 65,535 ≈ 15.3 → we need a factor ≥ 16 = 2^4, so **shift S = 4** (maximum window 1,048,560 bytes).

Problem 5 — Tracing sequence and acknowledgment numbers

A client with ISN (Initial Sequence Number) = 1000 connects to a server with ISN (Initial Sequence Number) = 5000. After the handshake the client sends 200 bytes, then 300 bytes; the server acknowledges both with one ACK (Acknowledgment), then sends a 500-byte reply. The client acknowledges the reply and closes the connection, followed by the server. Fill in the flags, sequence and acknowledgment numbers of every segment.

Table 11. *Solution of Problem 5*

#	Direction	Flags	seq	ack	Data (bytes)
1	C → S	SYN (Synchronize)	1000	—	0
2	S → C	SYN (Synchronize), ACK (Acknowledgment)	5000	1001	0
3	C → S	ACK (Acknowledgment)	1001	5001	0
4	C → S	ACK (Acknowledgment), PSH (Push)	1001	5001	200
5	C → S	ACK (Acknowledgment), PSH (Push)	1201	5001	300
6	S → C	ACK (Acknowledgment)	5001	1501	0
7	S → C	ACK (Acknowledgment), PSH (Push)	5001	1501	500
8	C → S	ACK (Acknowledgment)	1501	5501	0
9	C → S	FIN (Finish), ACK (Acknowledgment)	1501	5501	0
10	S → C	ACK (Acknowledgment)	5501	1502	0

#	Direction	Flags	seq	ack	Data (bytes)
11	S → C	FIN (Finish), ACK (Acknowledgment)	5501	1502	0
12	C → S	ACK (Acknowledgment)	1502	5502	0

Notice that SYN (Synchronize) and FIN (Finish) each consume one sequence number, while pure ACKs (Acknowledgments) consume none. Segments 6 and 7 are often merged in practice (*piggy-backing* the ACK (Acknowledgment) on the data).

Problem 6 — Congestion window trace (TCP (Transmission Control Protocol) Reno)

A TCP (Transmission Control Protocol) Reno sender starts with cwnd = 1 MSS (Maximum Segment Size) and ssthresh = 16 MSS (Maximum Segment Size). Three duplicate ACKs (Acknowledgments) are received at the end of round 9 and a timeout occurs at the end of round 18 (see Figure 6). Answer: (a) In which rounds does slow start operate? (b) What are ssthresh and cwnd after each loss event? (c) How many segments are sent in rounds 0–4? (d) What is cwnd in round 22?

Table 12. *cwnd per round (in MSS (Maximum Segment Size))*

Round	0	1	2	3	4	5	…	9	10	…	18	19	20	21	22	23
cwnd	1	2	4	8	16	17	…	21	10	…	18	1	2	4	8	9

Solution

(a) Slow start: rounds 0–4 (cwnd 1→16 reaches ssthresh) and rounds 19–22 after the timeout.

(b) After 3 dup-ACKs (Acknowledgments) at cwnd = 21: ssthresh = $\lfloor 21/2 \rfloor = 10$, cwnd = 10 (fast recovery, then congestion avoidance). After the timeout at cwnd = 18: ssthresh = 9, cwnd = 1.

(c) 1 + 2 + 4 + 8 + 16 = **31 segments**.

(d) Round 22: cwnd = 8 (slow start doubling 1, 2, 4, 8); in round 23 doubling would give 16 > ssthresh, so cwnd is capped at 9 and congestion avoidance begins.

Problem 7 — Computing the RTO (Retransmission Timeout)

Using RFC (Request for Comments) 6298 ($\alpha = 1/8$, $\beta = 1/4$, granularity G negligible), compute SRTT (Smoothed Round-Trip Time), RTTVAR (Round-Trip Time Variation) and RTO (Retransmission Timeout) after the RTT (Round-Trip Time) samples 100 ms and then 140 ms.

Solution

Sample 1 (100 ms): SRTT (Smoothed Round-Trip Time) = 100, RTTVAR (Round-Trip Time Variation) = 50, RTO (Retransmission Timeout) = 100 + 4·50 = **300 ms**.

Sample 2 (140 ms): RTTVAR (Round-Trip Time Variation) = 0.75·50 + 0.25·|100 − 140| = 37.5 + 10 = 47.5 ms; SRTT (Smoothed Round-Trip Time) = 0.875·100 + 0.125·140 = 105 ms; RTO (Retransmission Timeout) = 105 + 4·47.5 = **295 ms**.

Note: RFC (Request for Comments) 6298 recommends a lower bound of 1 s for RTO (Retransmission Timeout); Linux uses a minimum of 200 ms. With the 1 s bound, both values would be rounded up to 1 s.

Problem 8 — Decoding headers from a hex dump

(a) Decode the UDP (User Datagram Protocol) header `C3 50 00 35 00 2C 1A 2B`. (b) Decode the first 16 bytes of a TCP (Transmission Control Protocol) header: `D4 31 01 BB 00 00 03 E8 00 00 00 00 A0 02 FA F0`.

> **Solution**
>
> (a) Source port 0xC350 = **50000**; destination port 0x0035 = **53 (DNS (Domain Name System))**; length 0x002C = **44 bytes** → 36 bytes of payload; checksum 0x1A2B. This is a DNS (Domain Name System) query from an ephemeral port.
>
> (b) Source port 0xD431 = **54321**; destination port 0x01BB = **443 (HTTPS (Hypertext Transfer Protocol Secure))**; sequence number 0x000003E8 = **1000**; acknowledgment number 0 (not valid yet); data offset 0xA = 10 words = **40 bytes** (20 bytes of options); flags 0x02 = **SYN (Synchronize)**; window 0xFAF0 = **64,240 bytes**. This is the first segment of a three-way handshake.

Proposed exercises (homework)

1. Explain why DNS (Domain Name System) normally uses UDP (User Datagram Protocol) but switches to TCP (Transmission Control Protocol) for large responses and zone transfers.
2. A satellite link has R = 50 Mbit/s and RTT (Round-Trip Time) = 600 ms; frames are 1500 bytes. Compute the stop-and-wait utilisation and the window size (in frames and in bytes) for full utilisation.
3. A Go-Back-N sender with W = 4 sends frames 0–6; frame 2 is lost. List all frames transmitted until frame 6 is acknowledged. Repeat for Selective Repeat.
4. Draw the time diagram of a connection attempt to a closed port. Which segments appear? What does the client application see?
5. Explain the difference between CLOSE_WAIT and TIME_WAIT and say which side of the connection is in each state.
6. Starting with cwnd = 2 MSS (Maximum Segment Size) and ssthresh = 12 MSS (Maximum Segment Size), trace cwnd for 15 rounds if a timeout occurs at the end of round 8 (Tahoe and Reno behave the same here).
7. Why might a UDP (User Datagram Protocol)-based video application suffer less from a single lost packet than a TCP (Transmission Control Protocol)-based one? What could it lose instead?

Part III — Laboratory

The laboratory has two equivalent tracks with the same learning outcomes. Students work on the track that matches their lab machine (or do both if time allows). Each track takes about 100 minutes. Write your observations, command outputs (screenshots or copied text) and answers in the lab report (template at the end).

Table 13. *Map of laboratory tasks for both tracks*

Task	Goal	Windows tools	Linux tools
1	List open ports and connections	netstat, Get-NetTCPConnection, Get-NetUDPEndpoint	ss, lsof
2	Port number registry and ephemeral range	services file, netsh	/etc/services, sysctl
3	Test TCP (Transmission Control Protocol) reachability	Test-NetConnection	nc -zv
4	Capture the three-way handshake	Wireshark (Npcap) or pktmon	tcpdump, Wireshark
5	Local TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) exchange, TCP (Transmission Control Protocol) states	ncat or PowerShell .NET sockets	nc (netcat)
6	TCP (Transmission Control Protocol) settings and congestion control	netsh, Get-NetTCPSetting, netstat -s	sysctl, ss -ti
7	Throughput under delay and loss	iperf3 (two PCs (Personal Computers))	tc netem + iperf3

Safety and etiquette

- Only scan, capture or test hosts that you own or that the instructor designated for the lab.
- Remove every temporary firewall rule and traffic-control setting at the end of the session.
- Packet captures may contain passwords and personal data: do not share them outside the course.

Lab 6-W: Transport layer on Windows (Windows 10/11)

Prerequisites

- Windows 10 (version 2004 or later) or Windows 11; Windows PowerShell 5.1 or PowerShell 7.
- Wireshark with the **Npcap** driver, installed with the option *Support loopback traffic capture* (`winget install WiresharkFoundation.Wireshark`).
- Optional: **ncat** (installed with Nmap, `winget install Insecure.Nmap`) and **iperf3**.
- `curl.exe` is built into Windows 10/11. Some commands require *Run as administrator* — this is indicated.

Task W1 — Listing ports and connections

Command Prompt / PowerShell

```
netstat -an                      # all TCP/UDP endpoints, numeric
netstat -ano                     # ... plus owning process ID (PID)
netstat -anob                    # ... plus executable name (administrator)
netstat -an | findstr LISTENING  # only listening TCP sockets
```

PowerShell

```
Get-NetTCPConnection | Sort-Object State |
  Format-Table LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess -AutoSize

Get-NetTCPConnection -State Listen | Select-Object LocalAddress,LocalPort,OwningProcess
Get-NetUDPEndpoint | Select-Object LocalAddress,LocalPort,OwningProcess
Get-Process -Id (Get-NetTCPConnection -LocalPort 135 -State Listen).OwningProcess
```

Questions W1

- Which TCP (Transmission Control Protocol) ports are in state LISTEN on your machine? Identify the process for three of them.
- Why do UDP (User Datagram Protocol) endpoints have no State column?
- Open a web page in a browser and run the command again. Which remote ports appear? Which local ports were used, and in which range are they?

Task W2 — Port registry and ephemeral port range

PowerShell

```
Get-Content C:\Windows\System32\drivers\etc\services | Select-String "^(ftp|ssh|domain|http|https|ntp)\s"
netsh int ipv4 show dynamicport tcp
netsh int ipv4 show dynamicport udp
```

Questions W2

- Which ports correspond to the services listed? Do they match Table 4 of the lecture?
- What is the ephemeral range on Windows? How many simultaneous outgoing connections to the same server port does it allow from one IP (Internet Protocol) address?

Task W3 — Testing TCP (Transmission Control Protocol) reachability

PowerShell

```
Test-NetConnection www.example.com -Port 443        # TcpTestSucceeded : True
Test-NetConnection 127.0.0.1 -Port 12345           # closed port -> False (RST received)
Test-NetConnection www.example.com -Port 443 -InformationLevel Detailed
```

A closed port on a reachable host answers the SYN (Synchronize) with an **RST (Reset)**, so the test fails quickly. A port filtered by a firewall gives no answer and the test fails only after a timeout. Compare the time taken by the two cases.

Task W4 — Capturing the three-way handshake with Wireshark

1. Start Wireshark, select the active network interface (Ethernet or Wi-Fi) and start capturing.
2. Apply the display filter `tcp.port == 80 && ip.addr == <server-IP>` — or simply `http || tcp.flags.syn == 1`.
3. In PowerShell, generate a plain HTTP (Hypertext Transfer Protocol) connection: `curl.exe -s -o NUL http://example.com` (find the IP (Internet Protocol) with `Resolve-DnsName example.com`).
4. Stop the capture. Locate the **SYN (Synchronize)**, **SYN-ACK (Synchronize-Acknowledgment)** and **ACK (Acknowledgment)** segments. In the packet details, expand *Transmission Control Protocol* and note the ports, sequence/ACK (Acknowledgment) numbers, flags, window and options (MSS (Maximum Segment Size), SACK_PERM (Selective Acknowledgment Permitted), WS (Window Scale)).
5. Disable relative sequence numbers (*Edit → Preferences → Protocols → TCP (Transmission Control Protocol) → Relative sequence numbers*) and observe the real ISNs (Initial Sequence Numbers).

6. Locate the FIN (Finish) segments that close the connection. Who closed first?
7. Open *Statistics → Flow Graph* (select *TCP (Transmission Control Protocol) Flows*) and save the diagram for your report.

Without Wireshark: built-in Packet Monitor (administrator)

```
pktmon filter remove
pktmon filter add -p 80
pktmon start --capture --pkt-size 0
curl.exe -s -o NUL http://example.com
pktmon stop
pktmon etl2pcap PktMon.etl -o handshake.pcapng
```

The resulting .pcapng file can be opened later in Wireshark on any machine.

Questions W4

- Write down x (client ISN (Initial Sequence Number)) and y (server ISN (Initial Sequence Number)). Verify that ack = x + 1 and ack = y + 1 in segments 2 and 3.
- What MSS (Maximum Segment Size) does each side announce? Explain the value from the MTU (Maximum Transmission Unit).
- What window scale factor is used? What is the actual receive window in the first data segment?
- How many segments are needed for termination in your capture: three or four? Why might it be three?

Task W5 — Local TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) exchange; TCP (Transmission Control Protocol) states

Variant A — with ncat. Open two PowerShell windows.

TCP (Transmission Control Protocol) with ncat

```
# Window 1 (server)             # Window 2 (client)
ncat -l 5000                    ncat 127.0.0.1 5000
# type lines in either window; they appear in the other
```

Observing states (third window)

```
netstat -ano | findstr :5000      # two ESTABLISHED entries (one per direction of view)
# close the CLIENT with Ctrl+C, then immediately:
netstat -ano | findstr :5000      # TIME_WAIT on the side that closed first
```

UDP (User Datagram Protocol) with ncat

```
# Window 1                       # Window 2
ncat -u -l 6000                 ncat -u 127.0.0.1 6000
Get-NetUDPEndpoint -LocalPort 6000
```

Variant B — without extra tools (PowerShell + .NET).

PowerShell TCP (Transmission Control Protocol) server and client

```
# Window 1: one-shot TCP server on port 5000
$l = [System.Net.Sockets.TcpListener]::new([System.Net.IPAddress]::Loopback, 5000)
$l.Start(); "listening..."
$c = $l.AcceptTcpClient()
$r = [System.IO.StreamReader]::new($c.GetStream()); "received: " + $r.ReadLine()
$c.Close(); $l.Stop()

# Window 2: client
$c = [System.Net.Sockets.TcpClient]::new("127.0.0.1", 5000)
$w = [System.IO.StreamWriter]::new($c.GetStream()); $w.WriteLine("hello transport layer"); $w.Flush()
$c.Close()
```

Capture the exchange in Wireshark on the **Adapter for loopback traffic capture** with the filter `tcp.port == 5000 || udp.port == 6000`.

Questions W5

- How many packets are exchanged to send one line over TCP (Transmission Control Protocol; including set-up and tear-down)? And over UDP (User Datagram Protocol)?
- What happens if you send a UDP (User Datagram Protocol) datagram to port 6000 when no server is running? What does Wireshark show (look for ICMP (Internet Control Message Protocol))? What does ncat show?
- Which side ends up in TIME_WAIT? How long does the entry remain?

Task W6 — TCP (Transmission Control Protocol) settings and congestion control

PowerShell

```
netsh int tcp show global
Get-NetTCPSetting | Select-Object SettingName, CongestionProvider, AutoTuningLevelLocal,
InitialCongestionWindowMss
netstat -s -p tcp          # segments sent, received, retransmitted, connection resets
```

Questions W6

- Which congestion-control algorithm (CongestionProvider) is used for the *Internet* and *Datacenter* templates?
- What is the initial congestion window (in MSS (Maximum Segment Size))? Compare with the value in the lecture.
- What is *Receive Window Auto-Tuning*, and how does it relate to the Window Scale option you saw in Task W4?
- Compute the retransmission percentage from `netstat -s` (segments retransmitted / segments sent).

Task W7 (optional) — Throughput between two lab PCs (Personal Computers) with iperf3

PowerShell

```
# On PC 1 (server, administrator: allow the port temporarily)
New-NetFirewallRule -DisplayName "CN Lab iperf3" -Direction Inbound -Protocol TCP -LocalPort 5201 -Action
Allow -Profile Private
iperf3 -s

# On PC 2 (client)
iperf3 -c <PC1-IP> -t 10            # default window
iperf3 -c <PC1-IP> -t 10 -w 16K     # small window: throughput limited by W/RTT
iperf3 -c <PC1-IP> -t 10 -u -b 50M  # UDP at 50 Mbit/s: observe loss and jitter

# Clean-up on PC 1
Remove-NetFirewallRule -DisplayName "CN Lab iperf3"
```

Questions W7

- Compare the throughput with the default window and with a 16 KB window. Use ping to estimate the RTT (Round-Trip Time) and check the formula Throughput ≈ W / RTT (Round-Trip Time).
- In UDP (User Datagram Protocol) mode, what does iperf3 report that it cannot report for TCP (Transmission Control Protocol)?

Lab 6-L: Transport layer on Linux (Ubuntu 22.04/24.04 or Debian 12)

Prerequisites

Install the tools

```
sudo apt update
sudo apt install -y iproute2 tcpdump netcat-openbsd iperf3 curl dnsutils lsof wireshark
```

Commands marked with `sudo` need administrative rights. The netcat syntax below is for **netcat-openbsd** (the Ubuntu/Debian default); with *traditional* netcat use `nc -l -p 5000` instead of `nc -l 5000`.

Task L1 — Listing ports and connections

Terminal

```
ss -tuna                  # all TCP (-t) and UDP (-u) sockets, numeric (-n), all states (-a)
sudo ss -tlnp             # listening TCP sockets with process (-p)
sudo ss -ulnp             # UDP sockets with process
ss -s                     # summary statistics
ss -tan state established
sudo lsof -i :22          # which process uses port 22
```

Questions L1

- Which TCP (Transmission Control Protocol) ports are listening on your machine and which processes own them?
- Open a web page (`curl -s https://example.com > /dev/null` in a loop or a browser) and list the established connections. Which local port range do they use?

Task L2 — Port registry and ephemeral range

Terminal

```
grep -wE "^(ftp|ssh|domain|http|https|ntp)" /etc/services
getent services ssh
sysctl net.ipv4.ip_local_port_range        # default: 32768 60999
sysctl net.ipv4.ip_unprivileged_port_start  # ports below this need root (default 1024)
```

Task L3 — Testing TCP (Transmission Control Protocol) reachability

Terminal

```
nc -zv example.com 443        # "succeeded!" -> handshake completed
nc -zv 127.0.0.1 12345        # "Connection refused" -> RST received
nc -zv -w 3 10.255.255.1 80   # no answer -> timeout (filtered / unreachable)
```

Task L4 — Capturing the three-way handshake with tcpdump

Terminal

```
# Terminal 1: capture (-nn numeric, -S absolute sequence numbers)
sudo tcpdump -i any -nn -S 'tcp port 80 and host example.com'
# (run it again with  -w hs.pcap  to save the packets to a file for Wireshark)


# Terminal 2: generate a connection
curl -s -o /dev/null http://example.com
```

Typical output (shortened; addresses and numbers will differ)

```
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [S], seq 2954731205, win 64240,
   options [mss 1460,sackOK,TS val 1 ecr 0,nop,wscale 7], length 0
IP 93.184.215.14.80 > 10.0.2.15.41218: Flags [S.], seq 1270041366, ack 2954731206, win 65535,
   options [mss 1460,...], length 0
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [.], ack 1270041367, win 502, length 0
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [P.], seq 2954731206:2954731281, ack 1270041367, length 75:
HTTP: GET / HTTP/1.1
```

Table 14. *Flag notation in tcpdump*

tcpdump flag	Meaning
[S]	SYN (Synchronize)
[S.]	SYN (Synchronize) + ACK (Acknowledgment)
[.]	ACK (Acknowledgment) only
[P.]	PSH (Push) + ACK (Acknowledgment; data)
[F.]	FIN (Finish) + ACK (Acknowledgment)
[R] / [R.]	RST (reset)

Open `hs.pcap` in Wireshark (`wireshark hs.pcap`) to use the Flow Graph. Answer the same questions as in **Questions W4**.

Task L5 — Local TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) exchange; TCP (Transmission Control Protocol) states

TCP (Transmission Control Protocol) with nc

```
# Terminal 1 (server)          # Terminal 2 (client)
nc -l 5000                    nc 127.0.0.1 5000


# Terminal 3: observe
ss -tan '( sport = :5000 or dport = :5000 )'
sudo tcpdump -i lo -nn -S 'tcp port 5000'
# close the client (Ctrl+C), then:
ss -tan state time-wait '( sport = :5000 or dport = :5000 )'
```

UDP (User Datagram Protocol) with nc

```
# Terminal 1                    # Terminal 2
nc -u -l 6000                 nc -u 127.0.0.1 6000
ss -uan 'sport = :6000'
sudo tcpdump -i lo -nn 'udp port 6000 or icmp'
echo test | nc -u -w1 127.0.0.1 6001     # closed UDP port -> ICMP port unreachable
```

Questions L5

- Answer **Questions W5** for Linux.
- Compare the TIME_WAIT duration on Linux (about 60 s) with what you observed on Windows.

Task L6 — Sliding window, cwnd and congestion-control algorithms

Terminal

```
sysctl net.ipv4.tcp_congestion_control            # usually: cubic
cat /proc/sys/net/ipv4/tcp_available_congestion_control
sudo modprobe tcp_bbr && cat /proc/sys/net/ipv4/tcp_available_congestion_control
sysctl net.ipv4.tcp_window_scaling net.ipv4.tcp_sack net.ipv4.tcp_ecn
ss -ti dst example.com          # per-connection: cwnd, ssthresh, rtt, rto, mss, wscale
```

Keep a long download running (for example `curl -o /dev/null <URL of a large file>`, such as a Linux ISO (International Organization for Standardization) image from a mirror indicated by the instructor) and run `ss -ti` several times to watch **cwnd**, **rtt** and **rto** change.

Task L7 — Throughput under delay and loss with tc netem

Terminal

```
# Terminal 1
iperf3 -s


# Terminal 2: baseline on loopback
iperf3 -c 127.0.0.1 -t 10


# Emulate a WAN on the loopback interface: 50 ms delay each way, 1 % loss
sudo tc qdisc add dev lo root netem delay 50ms loss 1%
ping -c 3 127.0.0.1                     # RTT is now about 100 ms
iperf3 -c 127.0.0.1 -t 15 -i 1          # watch the Retr and Cwnd columns
iperf3 -c 127.0.0.1 -t 15 -i 1 -C reno  # same test with Reno
iperf3 -c 127.0.0.1 -t 15 -i 1 -C bbr   # same test with BBR (module loaded in L6)
sudo tc qdisc change dev lo root netem delay 50ms loss 5%


# ALWAYS remove the emulation at the end
sudo tc qdisc del dev lo root
```

Attention

The netem rule affects **all** loopback traffic of the machine (including local services). Do not forget the clean-up command `sudo tc qdisc del dev lo root`.

Questions L7

- Fill in a table: algorithm (cubic, reno, bbr) × loss (0 %, 1 %, 5 %) → average throughput and number of retransmissions.
- Plot the Cwnd column for one test. Can you recognise slow start and the reactions to loss?
- Why is the throughput of loss-based algorithms so sensitive to random loss, while BBR (Bottleneck Bandwidth and Round-trip propagation time) is less affected?

Lab report template

Table 15. *Structure of the lab report*

Section	Content
1. Identification	Name, group, date, track (Windows / Linux), OS (Operating System) version
2. Ports and connections	Outputs of Tasks 1–2 with the identified services and processes
3. Handshake analysis	Screenshot/flow graph; table with seq/ack/flags of the handshake and termination; MSS (Maximum Segment Size) and window scale
4. TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol) experiment	Packet counts, TIME_WAIT observation, behaviour of closed UDP/TCP (User Datagram Protocol/Transmission Control Protocol) ports
5. Congestion control	Algorithm in use, initial cwnd, measurements from Task 7 with a short interpretation
6. Conclusions	Three things you learned and one question that remains open

References

- J. F. Kurose, K. W. Ross — *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021 — Chapter 3.
- A. S. Tanenbaum, N. Feamster, D. Wetherall — *Computer Networks*, 6th ed., Pearson, 2021 — Chapter 6.
- RFC (Request for Comments) 768 (UDP (User Datagram Protocol)), RFC (Request for Comments) 9293 (TCP (Transmission Control Protocol)), RFC (Request for Comments) 6298 (RTO (Retransmission Timeout)), RFC (Request for Comments) 5681 (TCP (Transmission Control Protocol) congestion control), RFC (Request for Comments) 7323 (window scaling, timestamps), RFC (Request for Comments) 9438 (CUBIC), RFC (Request for Comments) 6335 (port registry).
- Wireshark User's Guide — wireshark.org/docs; man pages: ss(8), tcpdump(8), nc(1), tc-netem(8); Microsoft Learn: Get-NetTCPConnection, Test-NetConnection, pktmon.